



Porting Bluespec based IPs

SHAKTI Group | CSE Dept | PS-CDISHA - RISE LAB

IIT Madras

Agenda

1.	Introduction
2.	Existing peripherals & Adding New peripherals to SOC
3.	Classification of peripherals in Ganga & Yamuna SOC
4.	Flow of adding new peripheral into current project SOC
5.	Adding new (UART) peripheral into current project SOC

Introduction

- Flow of adding new or removing existing peripherals from <TARGET SOC>, based on end application needs for a better utilization of LUT's
- Example (Adding UART peripheral) procedure can be followed for the other peripherals to add or remove
- All the changes should be done according to the below mentioned path
 - `cd gc2025/hw/boards/ " < arty_a7_ganga> or <arty_a7_yamuna> "`

Existing Peripherals in both Ganga & Yamuna SOC

Ganga SOC

- 256 MB of DDR and 8 KB of Boot ROM
- 16 GPIOs
- 2 x GPTs
- 2 x SPIs
- 2 x UARTs
- 1 x I2C
- 1 x PLIC
- 1 x CLINT
- Onboard FTDI based Debugger
- Ethernet Lite IP from Xilinx

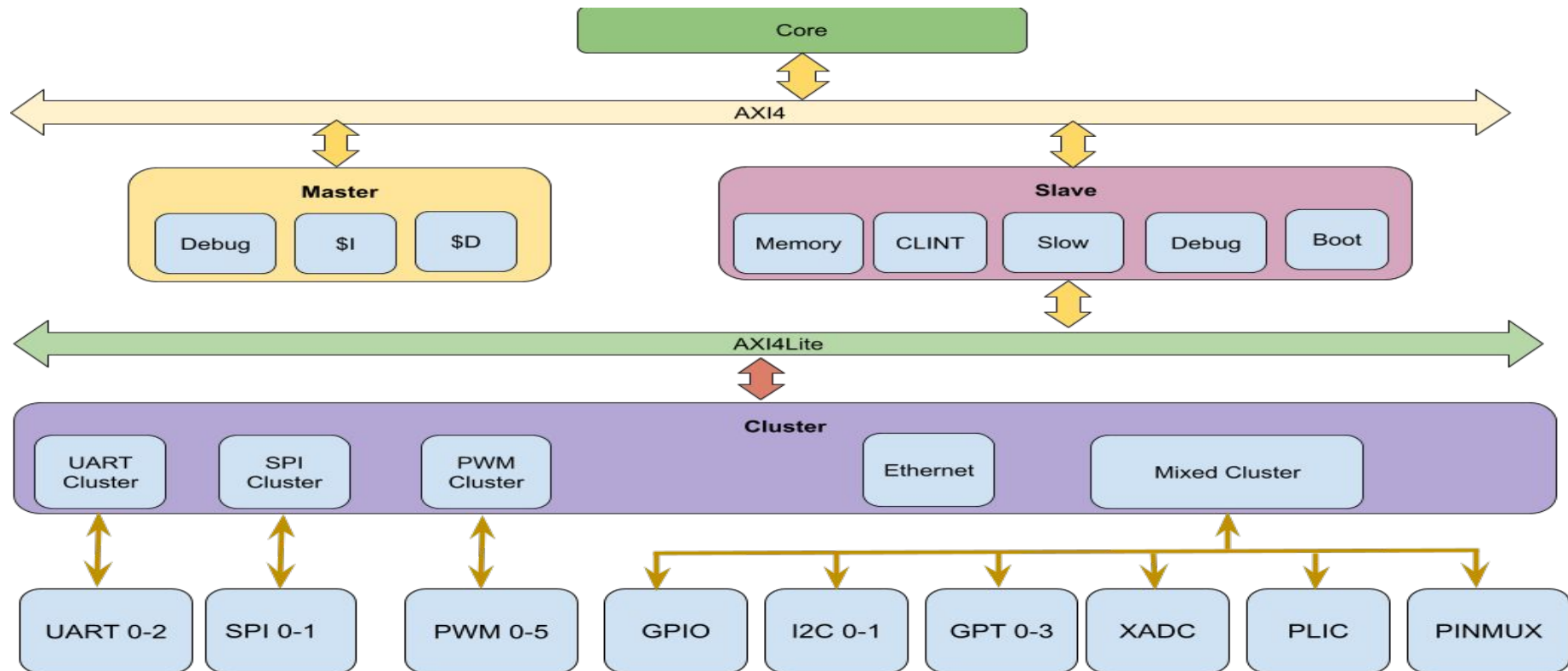
Yamuna SOC

- 256 MB of DDR and 8 KB of Boot ROM
- 32 x GPIOs
- 6 x PWMs
- 4 x GPTs
- 2 x SPIs
- 3 x UARTs
- 2 x I2Cs
- 1 x PLIC
- 1 x CLINT
- 1 x On board XADC
- Onboard FTDI based Debugger
- Ethernet Lite IP from Xilinx

Reconfigurable Peripherals in both Ganga & Yamuna SOC

- In the both SOC's all the peripherals can be configured based on end application.
- Either these peripherals can be added or removed with the existing peripherals to the SOC.
 - GPIO
 - GPT
 - UART
 - I2C
 - PINMUX
 - SPI
 - PWM
 - PLIC

Classification of peripherals in Yamuna SOC



Flow of adding new peripheral into current project SOC

- soc.defines
- <target>_cluster.bsv
- Soc.bsv
- Incrementing PLIC signals
- fpga_top.v
- constraints.xdc

Adding new UART peripheral into current project SOC

- Soc.defines

1. Add Cluster Base and End address for the UART.

```
68 `define UARTClusterBase 'h0001_1300
69 `define UARTClusterEnd 'h0001_1540
```



```
68 `define UARTClusterBase 'h0001_1300
69 `define UARTClusterEnd 'h0001_1640
70 `define SPIClusterBase 'h0002_0000
71 `define SPIClusterEnd 'h0002_01FF
```

2. Increment UART Cluster Slave Number

```
104 // - UART cluster
105 `define UARTCluster_Num_Slaves 4
106 `define UARTCluster_Num_Masters 1
```



```
104 // - UART cluster
105 `define UARTCluster_Num_Slaves 5
106 `define UARTCluster_Num_Masters 1
107
```

Adding new UART peripheral into current project SOC

3. Defining new UART3_slave_num as shown in below

```
108 `define UART0_slave_num 0
109 `define UART1_slave_num 1
110 `define UART2_slave_num 2
111 `define UARTCluster_err_slave_num 3
```



```
108 `define UART0_slave_num 0
109 `define UART1_slave_num 1
110 `define UART2_slave_num 2
111 `define UART3_slave_num 3
112 `define UARTCluster_err_slave_num 4
113
```

4. Defining new UART3 Base address and End address

```
113 `define UART0Base      'h0001_1300
114 `define UART0End      'h0001_1340
115 `define UART1Base      'h0001_1400
116 `define UART1End      'h0001_1440
117 `define UART2Base      'h0001_1500
118 `define UART2End      'h0001_1540
```



```
114 `define UART0Base      'h0001_1300
115 `define UART0End      'h0001_1340
116 `define UART1Base      'h0001_1400
117 `define UART1End      'h0001_1440
118 `define UART2Base      'h0001_1500
119 `define UART2End      'h0001_1540
120 `define UART3Base      'h0001_1600
121 `define UART3End      'h0001_1640
```

Adding new UART peripheral into current project SOC

- uart_cluster.bsv

1. Adding new UART3 Base and End address for decoding logic in uart_cluster.bsv

```
41 function Bit#(TLog#('UARTCluster_Num_Slaves)) fn_slave_map (Bit#('paddr) addr);
42   Bool slave_exist = True;
43   Bit#(TLog#('UARTCluster_Num_Slaves)) slave_num = 0;
44   if(   addr>= `UART0Base && addr<= `UART0End )
45     slave_num = `UART0_slave_num;
46   else if(addr>= `UART1Base && addr<= `UART1End )
47     slave_num = `UART1_slave_num;
48   else if(addr>= `UART2Base && addr<= `UART2End )
49     slave_num = `UART2_slave_num;
50   else
51     slave_num = `UARTCluster_err_slave_num;
52
53   return slave_num;
54 endfunction:fn_slave_map
```



```
41 function Bit#(TLog#('UARTCluster_Num_Slaves)) fn_slave_map (Bit#('paddr) addr);
42   Bool slave_exist = True;
43   Bit#(TLog#('UARTCluster_Num_Slaves)) slave_num = 0;
44   if(   addr>= `UART0Base && addr<= `UART0End )
45     slave_num = `UART0_slave_num;
46   else if(addr>= `UART1Base && addr<= `UART1End )
47     slave_num = `UART1_slave_num;
48   else if(addr>= `UART2Base && addr<= `UART2End )
49     slave_num = `UART2_slave_num;
50   * else if(addr>= `UART3Base && addr<= `UART3End )
51     slave_num = `UART3_slave_num; You, 14 seconds ago • Uncommitted change
52   else
53     slave_num = `UARTCluster_err_slave_num;
54
55   return slave_num;
56 endfunction:fn_slave_map
```

2. Declaring new RS232 uart3_io interface

```
56 interface Ifc_uart_cluster;
57   interface RS232 uart0_io;
58   interface RS232 uart1_io;
59   interface RS232 uart2_io;
60   interface AXI4_Lite_Slave_IFC#('paddr, 32, 0) slave;
61   method Bit#(3) uart_interrupts;
62 endinterface
```



```
58 interface Ifc_uart_cluster;
59   interface RS232 uart0_io;
60   interface RS232 uart1_io;
61   interface RS232 uart2_io;
62   * interface RS232 uart3_io;
63   interface AXI4_Lite_Slave_IFC#('paddr, 32, 0) slave;
64   method Bit#(3) uart_interrupts;
65 endinterface
```

Adding new UART peripheral into current project SOC

3. Instantiating another new uart3 inside “mkuart_cluster” module

```
85 let uart0 <- mkuart();  
86 let uart1 <- mkuart();  
87 let uart2 <- mkuart();
```



```
88 let uart0 <- mkuart();  
89 let uart1 <- mkuart();  
90 let uart2 <- mkuart();  
91 let uart3 <- mkuart();
```

4. Declaring the interface RS232 of uart3_io with uart3.io inside “mkuart_cluster” module

```
103 interface uart0_io=uart0.io;  
104 interface uart1_io=uart1.io;  
105 interface uart2_io=uart2.io;
```



```
108 interface uart0_io=uart0.io;  
109 interface uart1_io=uart1.io;  
110 interface uart2_io=uart2.io;  
111 interface uart3_io=uart3.io;
```

Adding new UART peripheral into current project SOC

5. Connecting new UART3 from AXI4_lite slave to AXI4_lite Master using mkconnection inside “mkuart_cluster” module

```
98      mkConnection (fabric.v_to_slaves [ `UART0_slave_num ],uart0.slave);  
99      mkConnection (fabric.v_to_slaves [ `UART1_slave_num ],uart1.slave);  
100     mkConnection (fabric.v_to_slaves [ `UART2_slave_num ],uart2.slave);
```



```
103     mkConnection (fabric.v_to_slaves [ `UART0_slave_num ],uart0.slave);  
104     mkConnection (fabric.v_to_slaves [ `UART1_slave_num ],uart1.slave);  
105     mkConnection (fabric.v_to_slaves [ `UART2_slave_num ],uart2.slave);  
106 | ✨ mkConnection (fabric.v_to_slaves [ `UART3_slave_num ],uart3.slave);
```

Adding new UART peripheral into current project SOC

- Soc.bsv

1. Declaring new “interface uart3_io RS232” inside “Ifc_chip_io” interface to take the new uart3 signals to out (on board)

```
120 interface Ifc_chip_io;  
121     interface Ifc_ssipi_io spi0_io;  
122     // interface Ifc_ssipi_io spi1_io;  
123     interface RS232 uart0_io;  
124         method I2C_out i2c0_out;  
125         method I2C_out i2c1_out;
```



```
120 interface Ifc_chip_io;  
121     interface Ifc_ssipi_io spi0_io;  
122     // interface Ifc_ssipi_io spi1_io;  
123     interface RS232 uart0_io;  
124     ✨ interface RS232 uart3_io;  
125     method I2C_out i2c0_out;  
126     method I2C_out i2c1_out;
```

2. Declaring the uart3_io interface instance which is declared in “mkuart_cluster” module to connect the new UART3 peripheral from uart_cluster.bsv to “Soc.bsv”

```
657 interface uart0_io = uart_cluster.uart0_io;  
658     method i2c0_out = mixed_cluster.i2c0_out;  
659     method i2c1_out = mixed_cluster.i2c1_out;
```



```
658 interface uart0_io = uart_cluster.uart0_io;  
659 ✨ interface uart3_io = uart_cluster.uart3_io;  
660     method i2c0_out = mixed_cluster.i2c0_out;  
661     method i2c1_out = mixed_cluster.i2c1_out;
```

Adding new UART peripheral into current project SOC

- uart_cluster.bsv

1. Incrementing uart_interrupt signals after adding a new UART3 peripheral, in method declaration, to pass new peripheral to PLIC

```
56 interface Ifc_uart_cluster;  
57     interface RS232 uart0_io;  
58     interface RS232 uart1_io;  
59     interface RS232 uart2_io;  
60     interface AXI4_Lite_Slave_IFC#(`paddr, 32, 0) slave;  
61     method Bit#(3) uart_interrupts;  
62 endinterface
```



```
63     interface AXI4_Lite_Slave_IFC#(`paddr, 32, 0) slave;  
64     method Bit#(4) uart_interrupts;  
65 endinterface
```

Adding new UART peripheral into current project SOC

- uart_cluster.bsv

2. Incrementing uart_interrupt signals in method declaration, inside mkuart_cluster declaration, to pass new peripheral to PLIC

```
103 interface uart0_io=uart0.io;
104 interface uart1_io=uart1.io;
105 interface uart2_io=uart2.io;
106 interface slave= c2s_xactor.axi_side;
107     method Bit#(3) uart_interrupts;
108         return {uart2.interrupt, uart1.interrupt, uart0.interrupt};
109         //return {uart2.interrupt, uart1.interrupt, uart0.interrupt};
110     endmethod
```



```
114 ✨ method Bit#(4) uart_interrupts;
115 ✨ return {uart3.interrupt, uart2.interrupt, uart1.interrupt, uart0.interrupt};
116 //return {uart2.interrupt, uart1.interrupt, uart0.interrupt};
117 endmethod
118 endmodule
119 endpackage
```

Adding new UART peripheral into current project SOC

- mixed_cluster.bsv

1. Incrementing interrupt signals in “lfc_mixed_cluster” interface declaration, in mixed_cluster.bsv to pass new peripheral to PLIC (UART is declared as external interrupt)

```
67 (*always_ready, always_enabled*)
68 method Action interrupts(Bit#(13) inp);
```



```
65 interface IOCellSide pinmuxtop_iocell_side;
66 interface PeripheralSide pinmuxtop_peripheral_side;
67 (*always_ready, always_enabled*)
68 ✨ method Action interrupts(Bit#(14) inp);
69 interface AXI4_Lite_Slave_IFC#(`paddr, 32, 0) slave;
70 interface AXI4_Lite_Master_IFC#(`paddr, 32, 0) xadc_master;
71 endinterface
```

Adding new UART peripheral into current project SOC

- mixed_cluster.bsv

2. Incrementing external_interrupt signals in “mkmixed_cluster” module, inside mixed_cluster.bsv to pass new peripheral to PLIC (UART is declared as external interrupt)

```
159 Ifc_err_slave_axi4lite#(`paddr, 32, 0 ) err_slave <- mkerr_slave_axi4lite;  
160     Wire#(Bit#(13)) wr_external_interrupts <- mkDWire('d0);  
161     Wire#(Bit#(2)) wr_sb_ext_interrupt <- mkDWire(0);
```



```
159 Ifc_err_slave_axi4lite#(`paddr, 32, 0 ) err_slave <- mkerr_slave_axi4lite;  
160     Wire#(Bit#(14)) wr_external_interrupts <- mkDWire('d0);  
161     Wire#(Bit#(2)) wr_sb_ext_interrupt <- mkDWire(0);
```

Adding new UART peripheral into current project SOC

- mixed_cluster.bsv

3. Incrementing external_interrupt signals in method declaration, inside mixed_cluster.bsv to pass new peripheral to PLIC (UART is declared as external interrupt)

```
205 |         method Action interrupts(Bit#(13) inp);  
206 |             wr_external_interrupts<= inp;  
207 |         endmethod
```



```
205 | ✨ method Action interrupts(Bit#(14) inp);  
206 | | wr_external_interrupts<= inp;  
207 | endmethod
```

Adding new UART peripheral into current project SOC

- mixed_cluster.bsv

4. Incrementing PLIC signals in mkplic module, inside mixed_cluster.bsv to pass new peripheral to PLIC (UART is declared as external interrupt)

```
96 (*synthesize*)
97 module mkplic(Ifc_plic_axi4lite#(`paddr, 32, 0, 35, 2, 7));
98   let ifc();
99   mkplic_axi4lite#(`PLICBase)_temp(ifc);
100   return ifc;
101 endmodule
```



```
96 ✨ (*synthesize*)
97 module mkplic(Ifc_plic_axi4lite#(`paddr, 32, 0, 36, 2, 7));
98   let ifc();
99   mkplic_axi4lite#(`PLICBase)_temp(ifc);
100   return ifc;
101 endmodule
```

Adding new UART peripheral into current project SOC

- mixed_cluster.bsv

5. Incrementing PLIC signals in rule declaration of `rl_connect_plic_connections`, inside `mixed_cluster.bsv` to pass new peripheral to PLIC (UART is declared as external interrupt)

```
Bit#(35) plic_inputs= {wr_external_interrupts[12:6], i2c1.isint, i2c0.isint, gptimer3.sb_interrupt, gptimer2.sb_interrupt, plic.sb_frm_sources(plic_inputs);
```



```
169 rule rl_connect_plic_connections;
170   let tmp <- gpio.sb_gpio_to_plic.get;
171   Bit#(16) lv_gpio_intr= truncate(pack(tmp));
172   ✨ Bit#(36) plic_inputs= {wr_external_interrupts[9], wr_external_interrupts[13:10],
   wr_external_interrupts[8:6], i2c1.isint, i2c0.isint, gptimer3.sb_interrupt, gptimer2.
   sb_interrupt, gptimer1.sb_interrupt, gptimer0.sb_interrupt, lv_gpio_intr,
173   wr_external_interrupts[5:0]}; #3 2025-10-08 Not Committed Yet You, 6 seconds ago
174   plic.sb_frm_sources(plic_inputs);
   endrule
```

Adding new UART peripheral into current project SOC

- fpga_top.v

1. Declaring new “uart3_SIN & uart3_SOUT” IO ports in fpga_top.v to take the signals out onto the board

```
65    input    uart0_SIN,  
66    output   uart0_SOUT,
```



```
64    // ---- UART ports -----//  
65    input    uart0_SIN,  
66    output   uart0_SOUT,  
67  
68    ✨ input ..... uart3_SIN,  
69    ✨ output ..... uart3_SOUT,  
70
```

2. Connecting the declared new uart3 ports to the UART module in SHAKTI <TARGET>SOC

```
584    // UART port definitions  
585    .uart0_io_SIN(uart0_SIN),  
586    .uart0_io_SOUT(uart0_SOUT),
```



```
587    // UART port definitions  
588    .uart0_io_SIN(uart0_SIN),  
589    .uart0_io_SOUT(uart0_SOUT),  
590    ✨ .uart3_io_SIN(uart3_SIN),  
591    ✨ ..... .uart3_io_SOUT(uart3_SOUT),
```

Adding new UART peripheral into current project SOC

- constraints.xdc

- Mapping new “uart3_SIN & uart3_SOUT” IO ports in constraints.xdc in the available ports, if there is no available ports then find the original constraints for Arty A7 100t and connect the new uart3 ports.

```
8  ## Pmod Header JB
9  set_property -dict { PACKAGE_PIN E15  IOSTANDARD LVCMOS33 } [get_ports { i2c0_sda }]; #IO_L11P_T1_SRCC_15 Sch=jb_p[1]
10 set_property -dict { PACKAGE_PIN E16  IOSTANDARD LVCMOS33 } [get_ports { i2c0_scl }]; #IO_L11N_T1_SRCC_15 Sch=jb_n[1]
11 set_property -dict { PACKAGE_PIN D15  IOSTANDARD LVCMOS33 } [get_ports { gpio_14 }]; #IO_L12P_T1_MRCC_15 Sch=jb_p[2]
12 set_property PULLDOWN true [get_ports { gpio_14 }];
13 set_property -dict { PACKAGE_PIN C15  IOSTANDARD LVCMOS33 } [get_ports { gpio_15 }]; #IO_L12N_T1_MRCC_15 Sch=jb_n[2]
14 set_property PULLDOWN true [get_ports { gpio_15 }];
15 set_property -dict { PACKAGE_PIN J17  IOSTANDARD LVCMOS33 } [get_ports { uart3_SOUT }]; #IO_L23P_T3_F0E_B_15 Sch=jb_p[3]
16 set_property -dict { PACKAGE_PIN J18  IOSTANDARD LVCMOS33 } [get_ports { uart3_SIN }]; #IO_L23N_T3_FWE_B_15 Sch=jb_n[3]
17 set_property -dict { PACKAGE_PIN J15  IOSTANDARD LVCMOS33 } [get_ports { external_clk }]; #IO_L24N_T3_RS0_15 Sch=jb_n[4]
```

TCL-based Bitstream & MCS Generation

- For Ganga SoC
 - make build BOARD=arty_a7_ganga

- For Yamuna SoC
 - make build BOARD=arty_a7_yamuna XLEN_build=32

Queries ?

Thank You !
